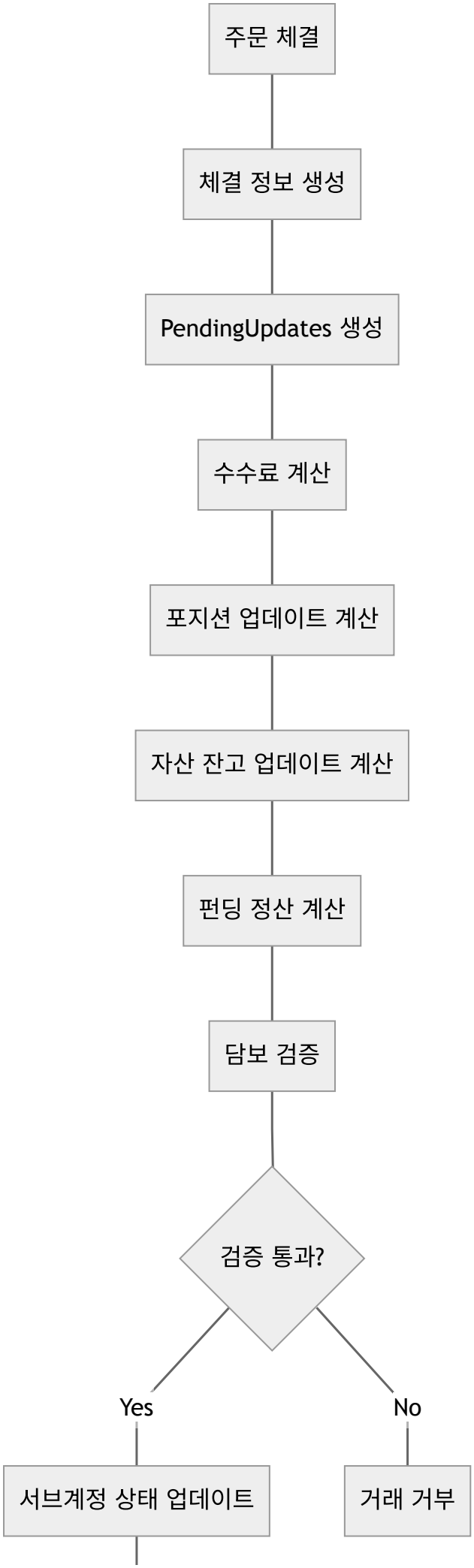
# dYdX v4 거래 처리 플로우 - 정산 및 포지션 관리 시스템 분석

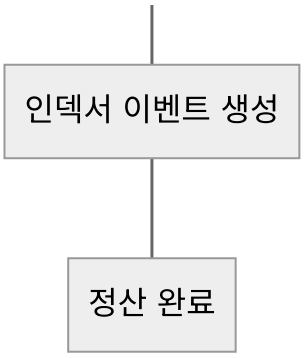
## 개요

이 문서는 dYdX v4의 정산 및 포지션 관리 시스템을 상세히 분석합니다. 체결 후 정산 프로세스 구현, 포지션 업데이트 및 마진 계산 로직, 그리고 수수료 계산 및 적용 방식을 포함합니다.

## 정산 시스템 아키텍처

### 전체 정산 플로우


주문 체결
체결 정보 생성
PendingUpdates 생성
수수료 계산
포지션 업데이트 계산
자산 잔고 업데이트 계산
펀딩 정산 계산
담보 검증
검증 통과?
Yes
No
서브계정 상태 업데이트
거래 거부




# 체결 후 정산 프로세스 구현 분석

## 1. 체결 정보 처리

### Fill 데이터 구조

```
type Fill struct {
// 체결 수량 (베이스 자산 단위)
FillAmount satypes.BaseQuantums
// Maker/Taker 주문 ID
MakerOrderId OrderId
TakerOrderId OrderId
// 수수료 (quote quantums 단위)
MakerFee int64
TakerFee int64
// 체결 가격 (subticks 단위)
Price Subticks
// 체결 블록 높이
BlockHeight uint32
}
```

### PendingUpdates를 통한 정산 집계

```
type PendingUpdates struct {
// 서브계정별 자산 업데이트 (assetId -> 수량 변화)
subaccountAssetUpdates map[satypes.SubaccountId]map[uint32]*big.Int
// 서브계정별 영구 선물 포지션 업데이트 (perpetualId -> 수량 변화)
subaccountPerpetualUpdates map[satypes.SubaccountId]map[uint32]*big.Int
// 서브계정별 총 수수료
subaccountFee map[satypes.SubaccountId]*big.Int
}

func NewPendingUpdates() *PendingUpdates {
return &PendingUpdates{
subaccountAssetUpdates: make(map[satypes.SubaccountId]map[uint32]*big.Int),
subaccountPerpetualUpdates: make(map[satypes.SubaccountId]map[uint32]*big.Int),
subaccountFee: make(map[satypes.SubaccountId]*big.Int),
}
}
```

### 영구 선물 체결 처리

```
func (p *PendingUpdates) AddPerpetualFill(
subaccountId satypes.SubaccountId,
perpetualId uint32,
```

```
isBuy bool,
feePpm int32,
bigFillBaseQuantums *big.Int,
bigFillQuoteQuantums *big.Int,
) {
// 1. 영구 선물 포지션 업데이트
if _, exists := p.subaccountPerpetualUpdates[subaccountId]; !exists {
p.subaccountPerpetualUpdates[subaccountId] = make(map[uint32]*big.Int)
}
perpetualUpdate, exists := p.subaccountPerpetualUpdates[subaccountId][perpetualI
if !exists {
perpetualUpdate = big.NewInt(0)
}
// 매수면 포지션 증가, 매도면 포지션 감소
if isBuy {
perpetualUpdate.Add(perpetualUpdate, bigFillBaseQuantums)
} else {
perpetualUpdate.Sub(perpetualUpdate, bigFillBaseQuantums)
}
p.subaccountPerpetualUpdates[subaccountId][perpetualId] = perpetualUpdate
// 2. Quote 자산 (USDC) 잔고 업데이트
if _, exists := p.subaccountAssetUpdates[subaccountId]; !exists {
p.subaccountAssetUpdates[subaccountId] = make(map[uint32]*big.Int)
```

```
}
usdcUpdate, exists := p.subaccountAssetUpdates[subaccountId][assettypes.AssetUsd
if !exists {
usdcUpdate = big.NewInt(0)
}
// 매수면 USDC 감소, 매도면 USDC 증가
if isBuy {
usdcUpdate.Sub(usdcUpdate, bigFillQuoteQuantums)
} else {
usdcUpdate.Add(usdcUpdate, bigFillQuoteQuantums)
}
p.subaccountAssetUpdates[subaccountId][assettypes.AssetUsdc.Id] = usdcUpdate
// 3. 수수료 계산 및 적용
totalFee, exists := p.subaccountFee[subaccountId]
if !exists {
totalFee = big.NewInt(0)
}
// 수수료 = 체결 금액 * 수수료율 (ppm 단위)
bigFeeQuoteQuantums := lib.BigIntMulSignedPpm(bigFillQuoteQuantums, feePpm, true
totalFee.Add(totalFee, bigFeeQuoteQuantums)
p.subaccountFee[subaccountId] = totalFee
}
```

## 2. 서브계정 업데이트 생성

### Update 구조체 생성

```
func (p *PendingUpdates) GetSubaccountUpdates() []satypes.Update {
var updates []satypes.Update
// 모든 영향받은 서브계정에 대해 업데이트 생성
for subaccountId := range p.subaccountPerpetualUpdates {
update := satypes.Update{
SubaccountId: subaccountId,
}
// 자산 업데이트 추가
if assetUpdates, exists := p.subaccountAssetUpdates[subaccountId]; exists {
for assetId, quantumsDelta := range assetUpdates {
// 수수료 차감
if assetId == assettypes.AssetUsdc.Id {
if fee, feeExists := p.subaccountFee[subaccountId]; feeExists {
quantumsDelta.Sub(quantumsDelta, fee)
}
}
update.AssetUpdates = append(update.AssetUpdates, satypes.AssetUpdate{
AssetId: assetId,
BigQuantumsDelta: quantumsDelta,
})
```

```
}

}

// 영구 선물 포지션 업데이트 추가

if perpUpdates, exists := p.subaccountPerpetualUpdates[subaccountId]; exists {

for perpetualId, quantumsDelta := range perpUpdates {

update.PerpetualUpdates = append(update.PerpetualUpdates, satypes.PerpetualUpdat

PerpetualId: perpetualId,

BigQuantumsDelta: quantumsDelta,

})

}

}

updates = append(updates, update)

}

return updates

}
```

# 포지션 업데이트 및 마진 계산 로직

## 1. 서브계정 업데이트 프로세스

### 메인 업데이트 함수

```
func (k Keeper) UpdateSubaccounts(

ctx sdk.Context,
```

```
updates []types.Update,

updateType types.UpdateType,

) (

success bool,

successPerUpdate []types.UpdateResult,

err error,

) {

// 1. 펀딩 정산이 포함된 업데이트 생성

settledUpdates, subaccountIdToFundingPayments, err := k.getSettledUpdates(ctx, u

if err != nil {

return false, nil, err

}

// 2. 모든 영구 선물 정보 가져오기

allPerps := k.perpetualsKeeper.GetAllPerpetuals(ctx)

// 3. 업데이트 가능 여부 검증 (담보, 마진 등)

success, successPerUpdate, err = k.internalCanUpdateSubaccounts(

ctx,

settledUpdates,

updateType,

allPerps,

)

if !success || err != nil {

return success, successPerUpdate, err
```

```
}

// 4. 영구 선물 펀딩 인덱스 맵 생성

perpIdToFundingIndex := make(map[uint32]dtypes.SerializableInt)

for _, perp := range allPerps {

perpIdToFundingIndex[perp.Params.Id] = perp.FundingIndex

}

// 5. 영구 선물 포지션 업데이트 적용

UpdatePerpetualPositions(settledUpdates, perpIdToFundingIndex)

// 6. 자산 포지션 업데이트 적용

UpdateAssetPositions(settledUpdates)

// 7. 상태 저장 및 이벤트 생성

for _, u := range settledUpdates {

k.SetSubaccount(ctx, u.SettledSubaccount)

// 인덱서 이벤트 생성

fundingPayments := subaccountIdToFundingPayments[*u.SettledSubaccount.Id]

k.GetIndexerEventManager().AddTxnEvent(

ctx,

indexerevents.SubtypeSubaccountUpdate,

indexerevents.SubaccountUpdateEventVersion,

indexer_manager.GetBytes(

indexerevents.NewSubaccountUpdateEvent(

u.SettledSubaccount.Id,

getUpdatedPerpetualPositions(u, fundingPayments),
```

```
getUpdatedAssetPositions(u),
fundingPayments,
),
),
)
// 펀딩 정산 이벤트 생성
sortedPerpIds := lib.GetSortedKeys[lib.Sortable[uint32]](fundingPayments)
for _, perpetualId := range sortedPerpIds {
fundingPaid := fundingPayments[perpetualId]
ctx.EventManager().EmitEvent(
types.NewCreateSettledFundingEvent(
*u.SettledSubaccount.Id,
perpetualId,
fundingPaid.BigInt(),
),
)
}
}
return success, successPerUpdate, err
}
```

## 2. 펀딩 정산 계산

### 펀딩 정산 로직

```
func (k Keeper) getSettledUpdates(
ctx sdk.Context,
updates []types.Update,
shouldApplyFunding bool,
) (
settledUpdates []settledUpdate,
subaccountIdToFundingPayments map[types.SubaccountId]map[uint32]dtypes.Serializa
err error,
) {
subaccountIdToFundingPayments = make(map[types.SubaccountId]map[uint32]dtypes.Se
for _, update := range updates {
// 현재 서브계정 상태 가져오기
subaccount := k.GetSubaccount(ctx, update.SubaccountId)
// 펀딩 정산 계산
settledSubaccount, fundingPayments, err := k.getSettledSubaccount(
ctx,
subaccount,
shouldApplyFunding,
)
if err != nil {
return nil, nil, err
}
```

```
// 업데이트 적용

newSubaccount := k.applyUpdatesToSubaccount(settledSubaccount, update)

settledUpdates = append(settledUpdates, settledUpdate{

SettledSubaccount: newSubaccount,

AssetUpdates: update.AssetUpdates,

PerpetualUpdates: update.PerpetualUpdates,

})

subaccountIdToFundingPayments[update.SubaccountId] = fundingPayments

}

return settledUpdates, subaccountIdToFundingPayments, nil

}
```

### 개별 서브계정 펀딩 정산

```
func (k Keeper) getSettledSubaccount(

ctx sdk.Context,

subaccount types.Subaccount,

shouldApplyFunding bool,

) (

settledSubaccount types.Subaccount,

fundingPayments map[uint32]dtypes.SerializableInt,

err error,

) {
```

```
fundingPayments = make(map[uint32]dtypes.SerializableInt)

if !shouldApplyFunding {

return subaccount, fundingPayments, nil

}

totalNetSettlementPpm := big.NewInt(0)

newPerpetualPositions := []*types.PerpetualPosition{}

// 각 영구 선물 포지션에 대해 펀딩 정산 계산

for _, position := range subaccount.PerpetualPositions {

perpetual, err := k.perpetualsKeeper.GetPerpetual(ctx, position.PerpetualId)

if err != nil {

return types.Subaccount{}, nil, err

}

// 펀딩 정산 계산

bigNetSettlementPpm, newFundingIndex := perpkeeper.GetSettlementPpmWithPerpetual

perpetual,

position.GetBigQuantums(),

position.FundingIndex.BigInt(),

)

// 펀딩 지불 기록 (정산의 음수)

if bigNetSettlementPpm.Cmp(lib.BigInt0()) != 0 {

fundingPayments[position.PerpetualId] = dtypes.NewIntFromBigInt(

new(big.Int).Neg(

new(big.Int).Div(bigNetSettlementPpm, lib.BigIntOneMillion()),
```

```
),
)
}
// 총 정산 금액 누적
totalNetSettlementPpm.Add(totalNetSettlementPpm, bigNetSettlementPpm)
// 펀딩 인덱스 업데이트
newPosition := *position
newPosition.FundingIndex = dtypes.NewIntFromBigInt(newFundingIndex)
newPerpetualPositions = append(newPerpetualPositions, &newPosition)
}
// USDC 잔고에 총 펀딩 정산 적용
newAssetPositions := []*types.AssetPosition{}
for _, assetPosition := range subaccount.AssetPositions {
newAssetPosition := *assetPosition
if assetPosition.AssetId == assettypes.AssetUsdc.Id {
// 펀딩 정산을 USDC 잔고에 반영
newAssetPosition.Quantums = dtypes.NewIntFromBigInt(
new(big.Int).Add(
assetPosition.Quantums.BigInt(),
new(big.Int).Div(totalNetSettlementPpm, lib.BigIntOneMillion()),
),
)
}
```

```
newAssetPositions = append(newAssetPositions, &newAssetPosition)

}

settledSubaccount = types.Subaccount{

Id: subaccount.Id,

AssetPositions: newAssetPositions,

PerpetualPositions: newPerpetualPositions,

MarginEnabled: subaccount.MarginEnabled,

}

return settledSubaccount, fundingPayments, nil

}
```

## 3. 마진 및 담보 검증

### 담보 충분성 검증

```
func (k Keeper) internalCanUpdateSubaccounts(

ctx sdk.Context,

settledUpdates []settledUpdate,

updateType types.UpdateType,

allPerps []perptypes.Perpetual,

) (

success bool,

successPerUpdate []types.UpdateResult,

err error,
```

```
) {
successPerUpdate = make([]types.UpdateResult, len(settledUpdates))
for i, update := range settledUpdates {
// 업데이트 후 서브계정 상태 시뮬레이션
updatedSubaccount := k.applyUpdatesToSubaccount(update.SettledSubaccount, update
// 담보 검증
risk, err := k.GetNetCollateralAndMarginRequirements(
ctx,
types.Update{
SubaccountId: *updatedSubaccount.Id,
AssetUpdates: update.AssetUpdates,
PerpetualUpdates: update.PerpetualUpdates,
},
)
if err != nil {
successPerUpdate[i] = types.NewlyUndercollateralized
continue
}
// 순 담보가 마진 요구사항보다 큰지 확인
if risk.NC.Cmp(risk.MMR) < 0 {
successPerUpdate[i] = types.NewlyUndercollateralized
continue
}
```

```
// 출금 제한 확인 (해당하는 경우)
if updateType == types.Withdrawal {
if risk.NC.Cmp(risk.IMR) < 0 {
successPerUpdate[i] = types.NewlyUndercollateralized
continue
}
}
successPerUpdate[i] = types.Success
}
// 모든 업데이트가 성공했는지 확인
success = true
for _, result := range successPerUpdate {
if result != types.Success {
success = false
break
}
}
return success, successPerUpdate, nil
}
```

# 수수료 계산 및 적용 방식 분석

## 1. 수수료 구조

### 수수료 타입

```
// 수수료는 ppm (parts per million) 단위로 계산
// 예: 500 ppm = 0.05% = 0.0005

type FeeCalculation struct {
// Maker 수수료 (일반적으로 음수 또는 0 - 리베이트)
MakerFeePpm int32
// Taker 수수료 (일반적으로 양수)
TakerFeePpm int32
// 청산 수수료
LiquidationFeePpm int32
}
```

## 2. 수수료 계산 로직

### 기본 수수료 계산

```
func CalculateFee(
fillQuoteQuantums *big.Int,
feePpm int32,
) *big.Int {
// 수수료 = 체결 금액 * 수수료율 (ppm)
// 양수 수수료는 올림, 음수 수수료는 내림 (0에 가깝게)
```

```
return lib.BigIntMulSignedPpm(fillQuoteQuantums, feePpm, true)

}
```

### 수수료 적용 예시

```
// 예시: 10,000 USDC 체결, Taker 수수료 500 ppm (0.05%)

fillAmount := big.NewInt(10_000_000_000) // 10,000 USDC (6자리 소수점)

takerFeePpm := int32(500)


fee := lib.BigIntMulSignedPpm(fillAmount, takerFeePpm, true)

// fee = 10,000,000,000 * 500 / 1,000,000 = 5,000,000 (5 USDC)


// Maker 리베이트 예시: -200 ppm (-0.02%)

makerFeePpm := int32(-200)

rebate := lib.BigIntMulSignedPpm(fillAmount, makerFeePpm, true)

// rebate = 10,000,000,000 * (-200) / 1,000,000 = -2,000,000 (-2 USDC, 즉 2 USDC
```

## 3. 수수료 등급 시스템

### 거래량 기반 수수료 등급

```
type FeeTier struct {

// 30일 거래량 기준 (USDC)

Volume *big.Int
```

```
// Maker 수수료율 (ppm)

MakerFeePpm int32

// Taker 수수료율 (ppm)

TakerFeePpm int32

}



// 예시 수수료 등급

var DefaultFeeTiers = []FeeTier{

{

Volume: big.NewInt(0), // 0 - 100K

MakerFeePpm: 200, // 0.02%

TakerFeePpm: 500, // 0.05%

},

{

Volume: big.NewInt(100_000_000_000), // 100K - 1M

MakerFeePpm: 100, // 0.01%

TakerFeePpm: 450, // 0.045%

},

{

Volume: big.NewInt(1_000_000_000_000), // 1M - 10M

MakerFeePpm: 0, // 0%

TakerFeePpm: 400, // 0.04%

},
```

```
{
Volume: big.NewInt(10_000_000_000_000), // 10M+
MakerFeePpm: -100, // -0.01% (리베이트)
TakerFeePpm: 350, // 0.035%
},
}
```

## 4. 수수료 분배

### 수수료 수취 및 분배

```
func (k Keeper) ProcessFees(
ctx sdk.Context,
fills []types.Fill,
) error {
totalFees := big.NewInt(0)
for _, fill := range fills {
// Maker 수수료 처리
if fill.MakerFee > 0 {
// 수수료 징수
err := k.CollectFee(ctx, fill.MakerOrderId.SubaccountId, fill.MakerFee)
if err != nil {
return err
}
```

```
totalFees.Add(totalFees, big.NewInt(fill.MakerFee))

} else if fill.MakerFee < 0 {

// 리베이트 지급

err := k.PayRebate(ctx, fill.MakerOrderId.SubaccountId, -fill.MakerFee)

if err != nil {

return err

}

totalFees.Sub(totalFees, big.NewInt(-fill.MakerFee))

}

// Taker 수수료 처리

if fill.TakerFee > 0 {

err := k.CollectFee(ctx, fill.TakerOrderId.SubaccountId, fill.TakerFee)

if err != nil {

return err

}

totalFees.Add(totalFees, big.NewInt(fill.TakerFee))

}

}

// 수수료를 프로토콜 수익으로 이전

if totalFees.Cmp(big.NewInt(0)) > 0 {

err := k.TransferFeesToProtocol(ctx, totalFees)

if err != nil {

return err
```

```
}

}

return nil

}
```

# 포지션 리스크 관리

## 1. 마진 요구사항 계산

### 초기 마진 (Initial Margin) 계산

```
func (k Keeper) GetInitialMarginRequirement(

ctx sdk.Context,

subaccountId types.SubaccountId,

) (*big.Int, error) {

subaccount := k.GetSubaccount(ctx, subaccountId)

totalIMR := big.NewInt(0)

for _, position := range subaccount.PerpetualPositions {

perpetual, err := k.perpetualsKeeper.GetPerpetual(ctx, position.PerpetualId)

if err != nil {

return nil, err

}

// 포지션 가치 계산

marketPrice, err := k.pricesKeeper.GetMarketPrice(ctx, perpetual.Params.MarketId
```

```
if err != nil {
return nil, err
}
positionNotional := new(big.Int).Mul(
position.GetBigQuantums().Abs(position.GetBigQuantums()),
marketPrice.Price.BigInt(),
)
// IMR = 포지션 가치 * 초기 마진 비율
imr := new(big.Int).Mul(
positionNotional,
big.NewInt(int64(perpetual.Params.InitialMarginPpm)),
)
imr.Div(imr, lib.BigIntOneMillion())
totalIMR.Add(totalIMR, imr)
}
return totalIMR, nil
}
```

### 유지 마진 (Maintenance Margin) 계산

```
func (k Keeper) GetMaintenanceMarginRequirement(
ctx sdk.Context,
subaccountId types.SubaccountId,
) (*big.Int, error) {
```

```
subaccount := k.GetSubaccount(ctx, subaccountId)
totalMMR := big.NewInt(0)
for _, position := range subaccount.PerpetualPositions {
perpetual, err := k.perpetualsKeeper.GetPerpetual(ctx, position.PerpetualId)
if err != nil {
return nil, err
}
// 포지션 가치 계산
marketPrice, err := k.pricesKeeper.GetMarketPrice(ctx, perpetual.Params.MarketId
if err != nil {
return nil, err
}
positionNotional := new(big.Int).Mul(
position.GetBigQuantums().Abs(position.GetBigQuantums()),
marketPrice.Price.BigInt(),
)
// MMR = 포지션 가치 * 유지 마진 비율
mmr := new(big.Int).Mul(
positionNotional,
big.NewInt(int64(perpetual.Params.MaintenanceMarginPpm)),
)
mmr.Div(mmr, lib.BigIntOneMillion())
totalMMR.Add(totalMMR, mmr)
```

```
}
return totalMMR, nil
}
```

## 2. 순 담보 계산

### 총 담보 가치 계산

```
func (k Keeper) GetNetCollateral(
ctx sdk.Context,
subaccountId types.SubaccountId,
) (*big.Int, error) {
subaccount := k.GetSubaccount(ctx, subaccountId)
netCollateral := big.NewInt(0)
// 자산 포지션 가치 합계
for _, assetPosition := range subaccount.AssetPositions {
asset, exists := k.assetsKeeper.GetAsset(ctx, assetPosition.AssetId)
if !exists {
continue
}
// 자산 가치를 USDC로 환산
assetValue := new(big.Int).Mul(
assetPosition.Quantums.BigInt(),
asset.DenomExponent.BigInt(),
```

```
)
netCollateral.Add(netCollateral, assetValue)
}
// 영구 선물 포지션의 미실현 손익 추가
for _, position := range subaccount.PerpetualPositions {
pnl, err := k.GetUnrealizedPnl(ctx, position)
if err != nil {
return nil, err
}
netCollateral.Add(netCollateral, pnl)
}
return netCollateral, nil
}
```

# 성능 최적화

## 1. 배치 처리

### 다중 업데이트 원자적 처리

```
func (k Keeper) BatchUpdateSubaccounts(
ctx sdk.Context,
updates []types.Update,
) error {
```

```
// 모든 업데이트를 하나의 트랜잭션으로 처리
success, _, err := k.UpdateSubaccounts(ctx, updates, types.Match)
if !success || err != nil {
return err
}
return nil
}
```

## 2. 캐싱 최적화

### 계산 결과 캐싱

```
type MarginCache struct {
subaccountId types.SubaccountId
blockHeight int64
imr *big.Int
mmr *big.Int
netCollateral *big.Int
}

func (k Keeper) GetCachedMarginRequirements(
ctx sdk.Context,
subaccountId types.SubaccountId,
) (*MarginCache, bool) {
```

```
// 블록 내에서 동일한 계산 결과 재사용

cacheKey := fmt.Sprintf("margin_%s_%d", subaccountId.String(), ctx.BlockHeight()

if cached, exists := k.marginCache[cacheKey]; exists {

return cached, true

}

return nil, false

}
```

## 결론

dYdX v4의 정산 및 포지션 관리 시스템은 복잡한 영구 선물 거래의 요구사항을 충족하기 위해 정교하게 설계되었습니다. 펀딩 정산, 수수료 계산, 마진 관리가 통합된 시스템을 통해 안전하고 효율적인 거래 환경을 제공합니다. 특히 원자적 업데이트와 엄격한 담보 검증을 통해 시스템의 안정성과 사용자 자금의 안전을 보장합니다.